Digital Signal Processing **HK261** **CO2036**

DIGITAL SIGNAL PROCESSING LAB REPORT

Lab 3 – Script Functions for Discrete-Time Signals

Âu Lê Quang Trí - Student ID: 2453300

Nguyễn Lê Khải Tú - Student ID: 2453384

October 14, 2026

Overview

This report presents the results of Lab 3 of the Digital Signal Processing laboratory. In this lab, the basic time-domain operations on discrete-time signals (delay, advance, folding, addition, multiplication and convolution) are implemented as reusable **Scilab script functions**. Every function receives a signal as a pair (`vector`, `origin position`), returns the resulting signal in the same format and graphically displays the input(s) and output in a single figure. The report also covers the additional exercises 2.1, 2.2, 2.4 and 2.6 of the reference document, both theoretically and practically on Scilab.

Table 1: Content of the report

Part	Content	Section
Exercise 1	`delay` – $y(n) = x(n-k)$	4.1
Exercise 2	`advance` – $y(n) = x(n+k)$	4.2
Exercise 3	`fold` – $y(n) = x(-n)$	4.3
Exercise 4	`add` – $y(n) = x_1(n) + x_2(n)$	4.4
Exercise 5	`multi` – $y(n) = x_1(n) \cdot x_2(n)$	4.5
Exercise 6	`convolution` – $y(n) = x(n) * h(n)$	4.6
Additional 2.1	Fold / delay of a signal	4.7
Additional 2.2	Sketching transformed signals	4.7
Additional 2.4	Even – odd decomposition	4.7
Additional 2.6	Time-invariance of systems	4.7

1 Introduction

Lab 1 and Lab 2 introduced Scilab and the elementary discrete-time signals, where each operation was written directly in the console for a particular signal. In Lab 3, these operations are generalized into script functions so that they can be applied to *any* finite-length sequence. The key difficulty is that a Scilab vector has no notion of the time index n: its elements are always indexed from 1. Therefore, each signal is represented by a vector `xn` together with an integer `xorigin` which stores the position of the sample $x(0)$ inside the vector. The time index of the element at position i is then

$$n_i = i - \texttt{xorigin}, \qquad i = 1, 2, \ldots, \texttt{length(xn)}. \tag{1}$$

2 Objectives

- Review the lecture notes on discrete-time signals and systems.
- Implement script-based manipulation functions in Scilab for discrete-time signals.
- Verify each function with an example and capture the output (console and figure) as evidence.
- Solve the additional exercises 2.1, 2.2, 2.4 and 2.6 both theoretically and on Scilab.

3 Methodology

3.1 Scilab script functions

A script function in Scilab is declared with the following format:

```
function [rv1, rv2, ..., rvn] = Function_Name(pv1, pv2, ..., pvn)
    // function body
endfunction
```

where `rv1, rv2, ...` are the return values and `pv1, pv2, ...` are the input parameters. Each function is stored in a `.sci` file, loaded with `exec('file.sci', -1)` and then called from the console.

3.2 Common conventions

All functions in this lab follow the same conventions:

- A signal is passed as a pair (`xn`, `xorigin`) and returned as (`yn`, `yorigin`).
- The time index vector is built from (1): `n = (1:length(xn)) - xorigin`.
- Signals are drawn with `plot2d3` (stems) plus circle markers, inside `subplots` of a single figure. All subplots share the same axis bounds so that the shift of the samples can be compared directly.

4 Practical Contents

4.1 Exercise 1 – Delay

Question: Write the function `[yn, yorigin] = delay(xn, xorigin, k)` that performs the delay operation $y(n) = x(n-k)$, where $k > 0$. $x(n)$ and $y(n)$ are graphically displayed in the same figure.

For example, $y(n) = x(n-1)$ with $x(n) = \{1, -2, \underset{\uparrow}{3}, 6\}$ is computed by `[yn, yorigin] = delay([1, -2, 3, 6], 3, 1)` and the output is `yn = [1, -2, 3, 6]`, `yorigin = 2`.

Explanation:

Explanation of the delay algorithm.

Source code:

Source code of `delay_lab3.sci`.

Result:

Screenshot of the console output and the figure of $x(n)$ and $y(n)$.

4.2 Exercise 2 – Advance

Question: Write the function `[yn, yorigin] = advance(xn, xorigin, k)` that performs the advance operation $y(n) = x(n + k)$, where $k > 0$. $x(n)$ and $y(n)$ are graphically displayed in the same figure.

Explanation: Advancing a signal by k samples moves every sample k positions to the *left* on the time axis: the value that was at index $n + k$ is now at index n. The sample values themselves, and their order, do not change. What changes is which sample is at the origin. In the (`xn`, `xorigin`) form this means:

- The new origin is k positions further to the right inside the vector:

$$\texttt{yorigin} = \texttt{xorigin} + k. \tag{2}$$

- If `yorigin` is still inside the vector, `yn = xn`. Otherwise, the origin falls after the last sample, so zeros are appended on the right until the vector reaches position `yorigin`. This keeps the sample $y(0)$ (which is 0) inside the vector.
- The input $k \leq 0$ is rejected with `error()`, because the operation is only defined for $k > 0$.

For the example $x(n) = \{1, -2, \underset{\uparrow}{3}, 6\}$ (`xorigin = 3`) and $k = 1$:

$$y(n) = x(n+1) = \{1, -2, 3, \underset{\uparrow}{6}\}, \qquad \texttt{yn} = [1, -2, 3, 6], \quad \texttt{yorigin} = 4. \tag{3}$$

Check: $y(0) = x(1) = 6$ and $y(-3) = x(-2) = 1$. So the whole sequence now occupies $-3 \leq n \leq 0$, one sample to the left of the original $-2 \leq n \leq 1$.

Source code:

```
function [yn, yorigin] = advance(xn, xorigin, k)
    // Check for valid k > 0
    if k <= 0 then
        error("k must be greater than 0");
```

```
    end

    // Time index vector for x(n)
    nx = (1:length(xn)) - xorigin;

    // Calculate new origin position
    yorigin = xorigin + k;

    // Pad zeros on the right if yorigin exceeds length of xn
    if yorigin > length(xn) then
        yn = [xn, zeros(1, yorigin - length(xn))];
    else
        yn = xn;
    end

    // Time index vector for y(n)
    ny = (1:length(yn)) - yorigin;

    // Common axis limits for clear comparison
    n_min = min(min(nx), min(ny)) - 1;
    n_max = max(max(nx), max(ny)) + 1;
    y_min = min([xn, yn, 0]) - 1;
    y_max = max([xn, yn, 0]) + 1;

    // Plot x(n) and y(n) in the same figure
    scf(); clf();

    subplot(2, 1, 1);
    plot2d3(nx, xn, style=2);
    plot(nx, xn, 'bo');
    xtitle("Original Signal x(n)", "n", "Amplitude");
    a1 = gca();
    a1.data_bounds = [n_min, y_min; n_max, y_max];
    xgrid();

    subplot(2, 1, 2);
    plot2d3(ny, yn, style=5);
    plot(ny, yn, 'ro');
    xtitle("Advanced Signal y(n) = x(n + " + string(k) + ")", "n", "Amplitude");
    a2 = gca();
    a2.data_bounds = [n_min, y_min; n_max, y_max];
    xgrid();
endfunction


[yn, yorigin] = advance([1, -2, 3, 6], 3, 1)
disp("yn = ", yn);
disp("yorigin = ", yorigin);
```

Listing 1: Function advance (advance_lab3.sci)

Result:

Console output

```
yn =
1.    -2.    3.    6.
yorigin =
4.
```

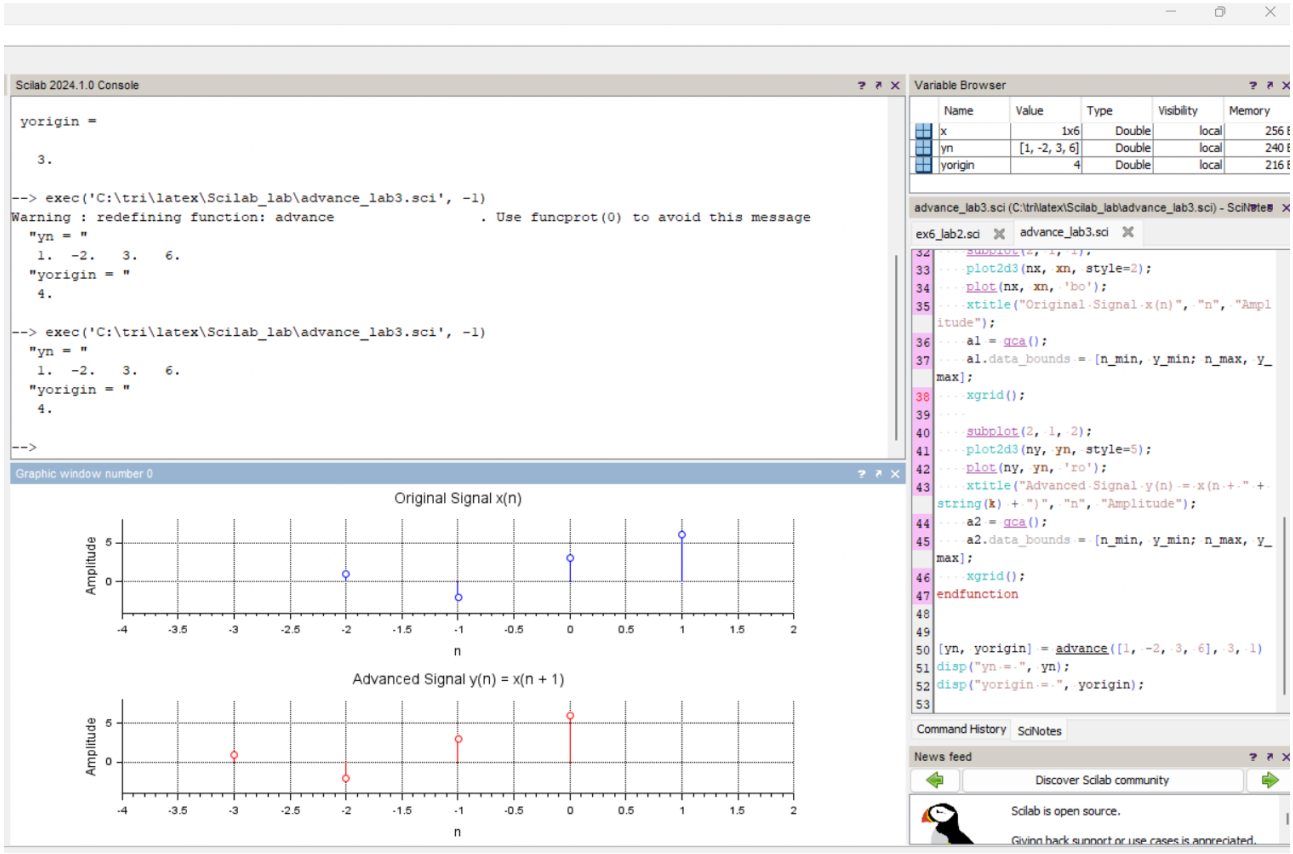


Figure 1: Result of Exercise 2 – `advance([1, -2, 3, 6], 3, 1)`: $x(n)$ (top) and $y(n) = x(n+1)$ (bottom)

The figure confirms the analysis: all four samples are shifted one unit to the left, and the sample 6 is now at $n = 0$.

4.3 Exercise 3 – Folding

Question: Write the function `[yn, yorigin] = fold(xn, xorigin)` that performs the folding operation $y(n) = x(-n)$. $x(n)$ and $y(n)$ are graphically displayed in the same figure.

Explanation:

Explanation of the folding algorithm.

Source code:

Source code of `fold_lab3.sci`.

Result:

Screenshot of the console output and the figure of $x(n)$ and $y(n)$.

4.4 Exercise 4 – Addition

Question: Write the function `[yn, yorigin] = add(x1n, x1origin, x2n, x2origin)` that performs the addition $y(n) = x_1(n) + x_2(n)$. $x_1(n)$, $x_2(n)$ and $y(n)$ are graphically displayed in the same figure.

For example, with $x_1(n) = \{\underset{\uparrow}{0}, 1, 3, -2\}$ and $x_2(n) = \{1, \underset{\uparrow}{1}, 2, 3\}$, the call `[yn, yorigin] = add([0, 1, 3, -2], 1, [1, 1, 2, 3], 2)` must return `yn = [1, 1, 3, 6, -2]` and `yorigin = 2`.

Explanation: Signals are added *sample by sample at the same time index*, not element by element in the vectors. If the two vectors have different lengths or different origins, they cannot be added directly with `+`. The function works in four steps:

1. Build the index vectors n_1 and n_2 of the two signals using (1).
2. Find the common time range that covers both signals:

$$n_{min} = \min(n_{1,min}, n_{2,min}), \qquad n_{max} = \max(n_{1,max}, n_{2,max}). \tag{4}$$

   The origin of the output is at position $\texttt{yorigin} = 1 - n_{min}$.
3. Zero-pad each signal on the left with $(n_{i,min} - n_{min})$ zeros and on the right with $(n_{max} - n_{i,max})$ zeros. After padding, both vectors have the same length and position i refers to the same time index in both.
4. Add the two padded vectors element-wise: `yn = x1_pad + x2_pad`.

For the example, $n_1 \in [0, 3]$ and $n_2 \in [-1, 2]$, so the common range is $[-1, 3]$:

n	-1	0	1	2	3
$x_1(n)$	0	0	1	3	-2
$x_2(n)$	1	1	2	3	0
$y(n)$	**1**	**1**	**3**	**6**	$\mathbf{-2}$

$$y(n) = \{1, \underset{\uparrow}{1}, 3, 6, -2\}, \qquad \texttt{yn} = [1, 1, 3, 6, -2], \quad \texttt{yorigin} = 1 - (-1) = 2, \tag{5}$$

which matches the expected output given in the assignment.

Source code:

```
function [yn, yorigin] = add(x1n, x1origin, x2n, x2origin)
    // Time index vectors for x1(n) and x2(n)
    n1 = (1:length(x1n)) - x1origin;
    n2 = (1:length(x2n)) - x2origin;

    // Determine common time range [n_min, n_max]
    n_min = min(min(n1), min(n2));
    n_max = max(max(n1), max(n2));
    ny = n_min : n_max;

    // New origin position (where n = 0 in ny)
    yorigin = 1 - n_min;

    // Pad x1n and x2n with zeros to match the length of ny
    x1_pad = [zeros(1, min(n1) - n_min), x1n, zeros(1, n_max - max(n1))];
    x2_pad = [zeros(1, min(n2) - n_min), x2n, zeros(1, n_max - max(n2))];

    // Perform addition
    yn = x1_pad + x2_pad;

    // Common axis bounds for plotting
    y_min = min([x1n, x2n, yn, 0]) - 1;
```

```
    y_max = max([x1n, x2n, yn, 0]) + 1;

    // Plot x1(n), x2(n), and y(n) in the same figure
    scf(); clf();

    subplot(3, 1, 1);
    plot2d3(n1, x1n, style=2);
    plot(n1, x1n, 'bo');
    xtitle("Signal x1(n)", "n", "Amplitude");
    a1 = gca();
    a1.data_bounds = [n_min - 1, y_min; n_max + 1, y_max];
    xgrid();

    subplot(3, 1, 2);
    plot2d3(n2, x2n, style=3);
    plot(n2, x2n, 'go');
    xtitle("Signal x2(n)", "n", "Amplitude");
    a2 = gca();
    a2.data_bounds = [n_min - 1, y_min; n_max + 1, y_max];
    xgrid();

    subplot(3, 1, 3);
    plot2d3(ny, yn, style=5);
    plot(ny, yn, 'ro');
    xtitle("Added Signal y(n) = x1(n) + x2(n)", "n", "Amplitude");
    a3 = gca();
    a3.data_bounds = [n_min - 1, y_min; n_max + 1, y_max];
    xgrid();
endfunction

//Example
[yn, yorigin] = add([0, 1, 3, -2], 1, [1, 1, 2, 3], 2)
disp("yn = ", yn);
disp("yorigin = ", yorigin);
```

Listing 2: Function `add` (`add_lap3.sci`)

Result:

Console output

```
yn =
1.    1.    3.    6.    -2.
yorigin =
2.
```

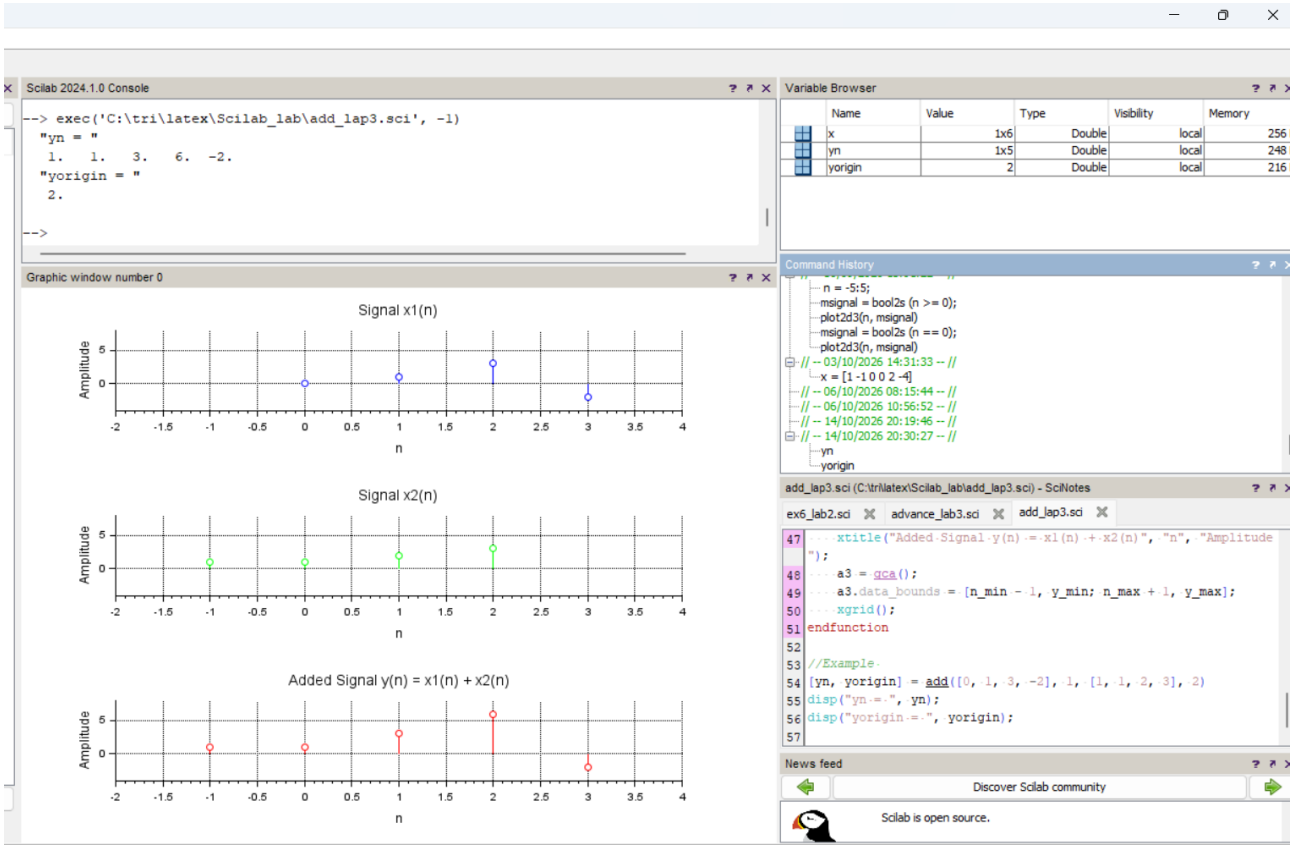


Figure 2: Result of Exercise 4 – `add([0, 1, 3, -2], 1, [1, 1, 2, 3], 2)`: $x_1(n)$, $x_2(n)$ and $y(n) = x_1(n) + x_2(n)$

4.5 Exercise 5 – Multiplication

Question: Write the function `[yn, yorigin] = multi(x1n, x1origin, x2n, x2origin)` that performs the multiplication $y(n) = x_1(n) \cdot x_2(n)$. $x_1(n)$, $x_2(n)$ and $y(n)$ are graphically displayed in the same figure.

Explanation:

Explanation of the multiplication algorithm.

Source code:

Source code of `multi_lab3.sci`.

Result:

Screenshot of the console output and the figure of $x_1(n)$, $x_2(n)$ and $y(n)$.

4.6 Exercise 6 – Convolution

Question: Write the function `[yn, yorigin] = convolution(xn, xorigin, hn, horigin)` that performs the convolution $y(n) = x(n) * h(n)$, where $x(n)$ is the input signal and $h(n)$ is the impulse response of the system.

Explanation: The output of an LTI system is the convolution sum of the input and the impulse response:

$$y(n) = x(n) * h(n) = \sum_{k=-\infty}^{\infty} x(k)\, h(n-k). \tag{6}$$

- **Sample values.** For finite sequences, the convolution sum is the same as multiplying two polynomials. Scilab computes it directly with `conv(xn, hn)`. If $x(n)$ has N samples and $h(n)$ has M samples, $y(n)$ has $N + M - 1$ samples.
- **Origin.** The first sample of $y(n)$ is at the sum of the first indices of the two inputs:

  $$n_{y,min} = n_{x,min} + n_{h,min} = (1 - \texttt{xorigin}) + (1 - \texttt{horigin}), \tag{7}$$

  so the origin of $y(n)$ is at position

  $$\texttt{yorigin} = 1 - n_{y,min} = \texttt{xorigin} + \texttt{horigin} - 1. \tag{8}$$

Example: $x(n) = \{1, \underset{\uparrow}{2}, 3, 1\}$ (`xorigin = 2`) and $h(n) = \{\underset{\uparrow}{1}, 2, 1, -1\}$ (`horigin = 1`). The output starts at $n = -1 + 0 = -1$ and has $4 + 4 - 1 = 7$ samples. Computing the convolution sum by hand:

$$
\begin{aligned}
y(-1) &= x(-1)h(0) = 1 \\
y(0) &= x(-1)h(1) + x(0)h(0) = 2 + 2 = 4 \\
y(1) &= x(-1)h(2) + x(0)h(1) + x(1)h(0) = 1 + 4 + 3 = 8 \\
y(2) &= x(-1)h(3) + x(0)h(2) + x(1)h(1) + x(2)h(0) = -1 + 2 + 6 + 1 = 8 \\
y(3) &= x(0)h(3) + x(1)h(2) + x(2)h(1) = -2 + 3 + 2 = 3 \\
y(4) &= x(1)h(3) + x(2)h(2) = -3 + 1 = -2 \\
y(5) &= x(2)h(3) = -1
\end{aligned}
$$

$$
y(n) = \{1, \underset{\uparrow}{4}, 8, 8, 3, -2, -1\}, \qquad \texttt{yorigin} = 2 + 1 - 1 = 2. \tag{9}
$$

Source code:

```
function [yn, yorigin] = convolution(xn, xorigin, hn, horigin)
    // Time index vectors for x(n) and h(n)
    nx = (1:length(xn)) - xorigin;
    nh = (1:length(hn)) - horigin;

    // Compute discrete convolution using Scilab's conv()
    yn = conv(xn, hn);//Length of  yn = M + N - 1

    // Calculate origin position of y(n)
    yorigin = xorigin + horigin - 1;
    ny = (1:length(yn)) - yorigin;

    // Common axis bounds for plotting
    n_min = min([min(nx), min(nh), min(ny)]) - 1;
    n_max = max([max(nx), max(nh), max(ny)]) + 1;
    y_min = min([xn, hn, yn, 0]) - 1;
    y_max = max([xn, hn, yn, 0]) + 1;

    // Plot x(n), h(n), and y(n) in the same figure
    scf(); clf();

    subplot(3, 1, 1);
    plot2d3(nx, xn, style=2);
    plot(nx, xn, 'bo');
    xtitle("Input Signal x(n)", "n", "Amplitude");
    a1 = gca();
    a1.data_bounds = [n_min, y_min; n_max, y_max];
    xgrid();

    subplot(3, 1, 2);
    plot2d3(nh, hn, style=3);
    plot(nh, hn, 'go');
    xtitle("Impulse Response h(n)", "n", "Amplitude");
```

```
    a2 = gca();
    a2.data_bounds = [n_min, y_min; n_max, y_max];
    xgrid();

    subplot(3, 1, 3);
    plot2d3(ny, yn, style=5);
    plot(ny, yn, 'ro');
    xtitle("Convolved Signal y(n) = x(n) * h(n)", "n", "Amplitude");
    a3 = gca();
    a3.data_bounds = [n_min, y_min; n_max, y_max];
    xgrid();
endfunction


// Let x(n) = {1, 2^, 3, 1} (xorigin = 2) and h(n) = {1^, 2, 1, -1} (horigin = 1)
[yn, yorigin] = convolution([1, 2, 3, 1], 2, [1, 2, 1, -1], 1)
disp("yn = ", yn);
disp("yorigin = ", yorigin);
```

Listing 3: Function `convolution` (`convolution_lap3.sci`)

Result:

Console output

```
yn =
1.    4.    8.    8.    3.    -2.    -1.
yorigin =
2.
```

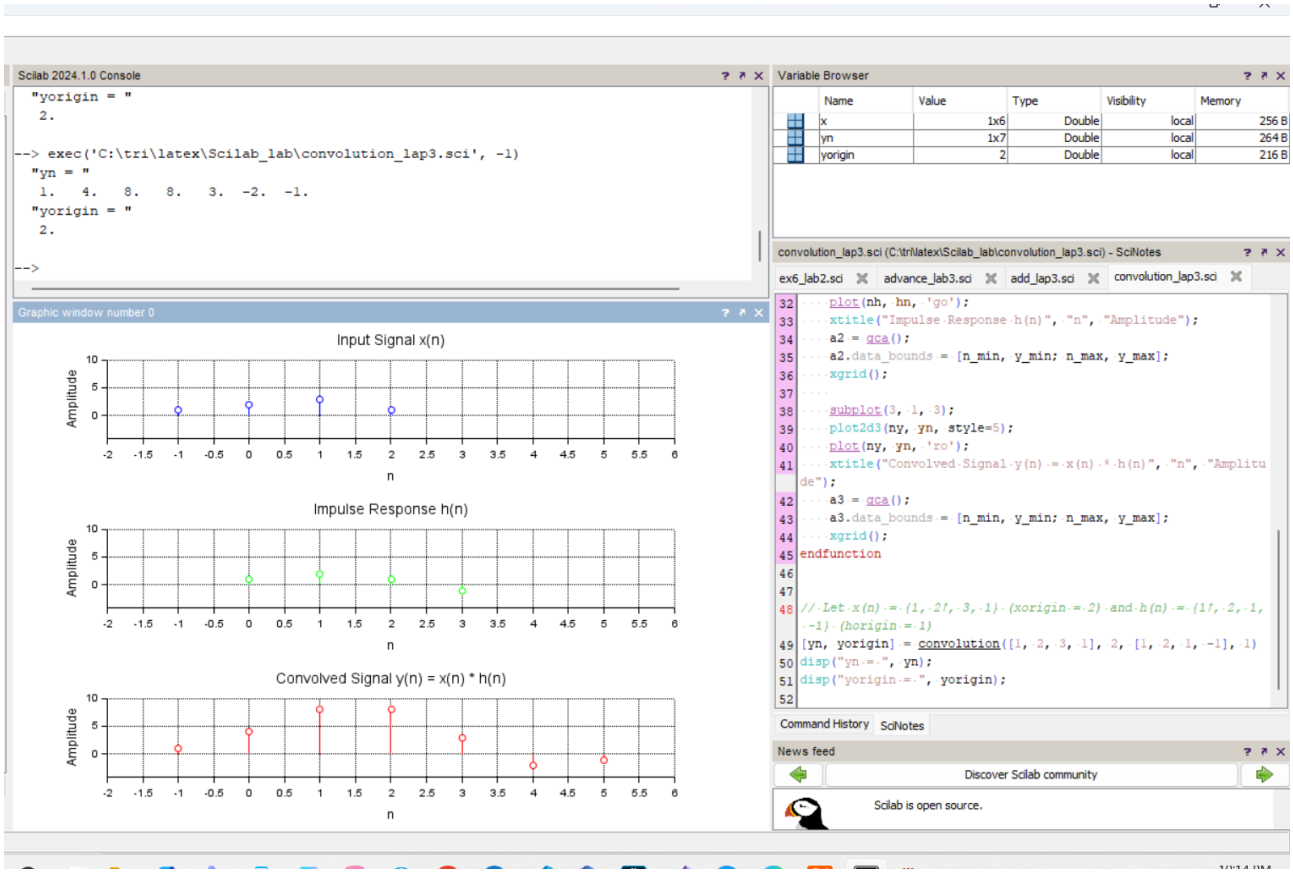


Figure 3: Result of Exercise 6 – `convolution([1, 2, 3, 1], 2, [1, 2, 1, -1], 1)`: $x(n)$, $h(n)$ and $y(n) = x(n) * h(n)$

The Scilab result matches the hand calculation exactly.

4.7 Additional Exercises

Additional exercise 2.1

A discrete-time signal $x(n)$ is defined as

$$x(n) = \begin{cases} 1 + \dfrac{n}{3}, & -3 \le n \le -1 \\ 1, & 0 \le n \le 3 \\ 0, & \text{elsewhere} \end{cases}$$

(a) Determine its values and sketch the signal $x(n)$.

(b) Sketch the signals that result if we:

1. First fold $x(n)$ and then delay the resulting signal by four samples.
2. First delay $x(n)$ by four samples and then fold the resulting signal.

(c) Sketch the signal $x(-n + 4)$.

(d) Compare the results in parts (b) and (c) and derive a rule for obtaining the signal $x(-n+k)$ from $x(n)$.

(e) Can you express the signal $x(n)$ in terms of signals $\delta(n)$ and $u(n)$?

Answer:

(a)

Answer (a)

(b)

Answer (b)

(c)

Answer (c)

(d)

Answer (d)

(e)

Answer (e)

Scilab:

Scilab source code for exercise 2.1.

Scilab output figure for exercise 2.1.

Additional exercise 2.2

A discrete-time signal $x(n)$ is shown in Fig. P2.2. Sketch and label carefully each of the following signals.

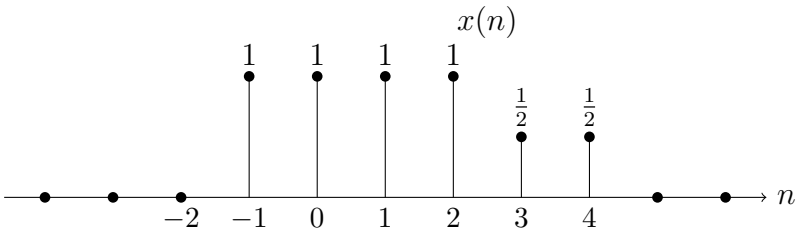


Figure 4: Figure P2.2 – the signal $x(n)$

(a) $x(n-2)$ **(b)** $x(4-n)$ **(c)** $x(n+2)$ **(d)** $x(n)u(2-n)$ **(e)** $x(n-1)\delta(n-3)$ **(f)** $x(n^2)$ **(g)** even part of $x(n)$ **(h)** odd part of $x(n)$

Answer: From the figure, $x(-1) = x(0) = x(1) = x(2) = 1$, $x(3) = x(4) = \frac{1}{2}$ and $x(n) = 0$ elsewhere:

$$x(n) = \{\ldots, 0, 1, \underset{\uparrow}{1}, 1, 1, \tfrac{1}{2}, \tfrac{1}{2}, 0, \ldots\}$$

(a) $x(n-2)$ is $x(n)$ *delayed* by 2 samples (shifted 2 units to the right). The support moves

from $[-1, 4]$ to $[1, 6]$:

$$x(n-2) = \{\underset{\uparrow}{0}, 1, 1, 1, 1, \tfrac{1}{2}, \tfrac{1}{2}\}$$

(b) $x(4-n) = x\big(-(n-4)\big)$: first fold $x(n)$ to get $x(-n) = \{\frac{1}{2}, \frac{1}{2}, 1, 1, \underset{\uparrow}{1}, 1\}$ (support $[-4, 1]$), then delay the folded signal by 4 samples (support $[0, 5]$):

$$x(4-n) = \{\underset{\uparrow}{\tfrac{1}{2}}, \tfrac{1}{2}, 1, 1, 1, 1\}$$

Check: $y(0) = x(4) = \frac{1}{2}$ and $y(5) = x(-1) = 1$.

(c) $x(n+2)$ is $x(n)$ *advanced* by 2 samples (shifted 2 units to the left). The support moves to $[-3, 2]$:

$$x(n+2) = \{1, 1, 1, \underset{\uparrow}{1}, \tfrac{1}{2}, \tfrac{1}{2}\}$$

(d) $u(2-n) = 1$ for $n \le 2$ and 0 for $n > 2$, so it keeps only the samples with $n \le 2$:

$$x(n)u(2-n) = \{1, \underset{\uparrow}{1}, 1, 1\}$$

(e) $\delta(n-3)$ is non-zero only at $n = 3$, so $x(n-1)\delta(n-3) = x(3-1)\,\delta(n-3) = x(2)\,\delta(n-3) = \delta(n-3)$:

$$x(n-1)\delta(n-3) = \{\underset{\uparrow}{0}, 0, 0, 1\}$$

(f) $x(n^2)$: $n = 0 \Rightarrow x(0) = 1$; $n = \pm 1 \Rightarrow x(1) = 1$; $n = \pm 2 \Rightarrow x(4) = \frac{1}{2}$; $|n| \ge 3 \Rightarrow n^2 \ge 9 \Rightarrow x(n^2) = 0$:

$$x(n^2) = \{\tfrac{1}{2}, 1, \underset{\uparrow}{1}, 1, \tfrac{1}{2}\}$$

(g)(h) The even and odd parts are $x_e(n) = \frac{x(n)+x(-n)}{2}$ and $x_o(n) = \frac{x(n)-x(-n)}{2}$:

n	-4	-3	-2	-1	0	1	2	3	4
$x(n)$	0	0	0	1	1	1	1	$\frac{1}{2}$	$\frac{1}{2}$
$x(-n)$	$\frac{1}{2}$	$\frac{1}{2}$	1	1	1	1	0	0	0
$x_e(n)$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{2}$	1	1	1	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{4}$
$x_o(n)$	$-\frac{1}{4}$	$-\frac{1}{4}$	$-\frac{1}{2}$	0	0	0	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{4}$

$$x_e(n) = \{\tfrac{1}{4}, \tfrac{1}{4}, \tfrac{1}{2}, 1, \underset{\uparrow}{1}, 1, \tfrac{1}{2}, \tfrac{1}{4}, \tfrac{1}{4}\}, \qquad x_o(n) = \{-\tfrac{1}{4}, -\tfrac{1}{4}, -\tfrac{1}{2}, 0, \underset{\uparrow}{0}, 0, \tfrac{1}{2}, \tfrac{1}{4}, \tfrac{1}{4}\}$$

We can check that $x_e(n) = x_e(-n)$, $x_o(n) = -x_o(-n)$ and $x_e(n) + x_o(n) = x(n)$.

Scilab: The signal $x(n)$ is written as a Scilab function `xsig(n)` that can be evaluated at *any* integer index vector. Each transformation is then obtained just by changing the argument, e.g. `xsig(4 - n)` or `xsig(n.*2)`, over a common index grid $n = -6, \ldots, 8$.

```
// ======================================================================
// Lab 3 - Additional Exercise 2.2
// x(n) from Figure P2.2:  x(n) = {1, 1^, 1, 1, 1/2, 1/2}   (^ = origin)
// i.e. x(n) = 1 for -1 <= n <= 2, x(3) = x(4) = 1/2, 0 elsewhere
// ======================================================================

// x(n) evaluated at ANY integer index vector n
function y = xsig(n)
    y = bool2s(n >= -1 & n <= 2) + 0.5 * bool2s(n == 3 | n == 4);
endfunction

// Unit step u(n) and unit sample delta(n)
function y = ustep(n)
    y = bool2s(n >= 0);
endfunction

function y = delta(n)
    y = bool2s(n == 0);
endfunction

// Stem plot with markers, grid and labels
function stem(n, x, ttl, c, spec)
    plot2d3(n, x, style=c);
    plot(n, x, spec);
    xtitle(ttl, "n", "Amplitude");
    a = gca();
    a.data_bounds = [min(n) - 0.5, min([x, 0]) - 0.25; max(n) + 0.5, max([x, 0]) +
    0.25];
    xgrid();
endfunction

// Print the signal as {v1, v2, v0*, ...} where * marks the origin n = 0
function show(name, n, x)
    k  = find(x <> 0);
    lo = min([n(k), 0]);
    hi = max([n(k), 0]);
    idx = find(n >= lo & n <= hi);
    s = string(x(idx));
    s(n(idx) == 0) = s(n(idx) == 0) + "*";
    mprintf("%-16s = {%s},   n = %d..%d\n", name, strcat(s, ", "), lo, hi);
endfunction

// ------------------------- Computation -----------------------------
n  = -6:8;
x  = xsig(n);
ya = xsig(n - 2);                    // (a) x(n-2)
yb = xsig(4 - n);                    // (b) x(4-n)
yc = xsig(n + 2);                    // (c) x(n+2)
yd = xsig(n) .* ustep(2 - n);        // (d) x(n)u(2-n)
```

```
ye = xsig(n - 1) .* delta(n - 3);     // (e) x(n-1)delta(n-3)
yf = xsig(n.^2);                      // (f) x(n^2)
yg = (xsig(n) + xsig(-n)) / 2;        // (g) even part
yh = (xsig(n) - xsig(-n)) / 2;        // (h) odd part

mprintf("\n----- Exercise 2.2 (* marks n = 0) -----\n");
show("x(n)", n, x);
show("(a) x(n-2)", n, ya);
show("(b) x(4-n)", n, yb);
show("(c) x(n+2)", n, yc);
show("(d) x(n)u(2-n)", n, yd);
show("(e) x(n-1)d(n-3)", n, ye);
show("(f) x(n^2)", n, yf);
show("(g) xe(n)", n, yg);
show("(h) xo(n)", n, yh);
mprintf("Check xe(n) + xo(n) = x(n): %s\n", string(and(yg + yh == x)));

// --------------------------- Plotting ------------------------------
f22 = scf(); clf();
f22.figure_size = [1300, 950];
subplot(3, 3, 1); stem(n, x,  "x(n)",                  2, "bo");
subplot(3, 3, 2); stem(n, ya, "(a) x(n-2)",            5, "ro");
subplot(3, 3, 3); stem(n, yb, "(b) x(4-n)",            5, "ro");
subplot(3, 3, 4); stem(n, yc, "(c) x(n+2)",            5, "ro");
subplot(3, 3, 5); stem(n, yd, "(d) x(n)u(2-n)",        5, "ro");
subplot(3, 3, 6); stem(n, ye, "(e) x(n-1)delta(n-3)",  5, "ro");
subplot(3, 3, 7); stem(n, yf, "(f) x(n^2)",            5, "ro");
subplot(3, 3, 8); stem(n, yg, "(g) even part xe(n)",   3, "go");
subplot(3, 3, 9); stem(n, yh, "(h) odd part xo(n)",    6, "mo");
```

Listing 4: Additional exercise 2.2 (`ex2_2_lab3.sce`)

Console output

```
(* marks the sample at n = 0)
x(n)                                    = {1, 1*, 1, 1, 0.5, 0.5},    n = -1..4
(a) x(n-2)                            = {0*, 1, 1, 1, 1, 0.5, 0.5},    n = 0..6
(b) x(4-n)                              = {0.5*, 0.5, 1, 1, 1, 1},    n = 0..5
(c) x(n+2)                             = {1, 1, 1, 1*, 0.5, 0.5},    n = -3..2
(d) x(n)u(2-n)                                   = {1, 1*, 1, 1},    n = -1..2
(e) x(n-1)d(n-3)                                  = {0*, 0, 0, 1},    n = 0..3
(f) x(n^2)                                 = {0.5, 1, 1*, 1, 0.5},    n = -2..2
(g) xe(n)             = {0.25, 0.25, 0.5, 1, 1*, 1, 0.5, 0.25, 0.25},    n = -4..4
(h) xo(n)          = {-0.25, -0.25, -0.5, 0, 0*, 0, 0.5, 0.25, 0.25},    n = -4..4
Check xe(n) + xo(n) = x(n):  T
```

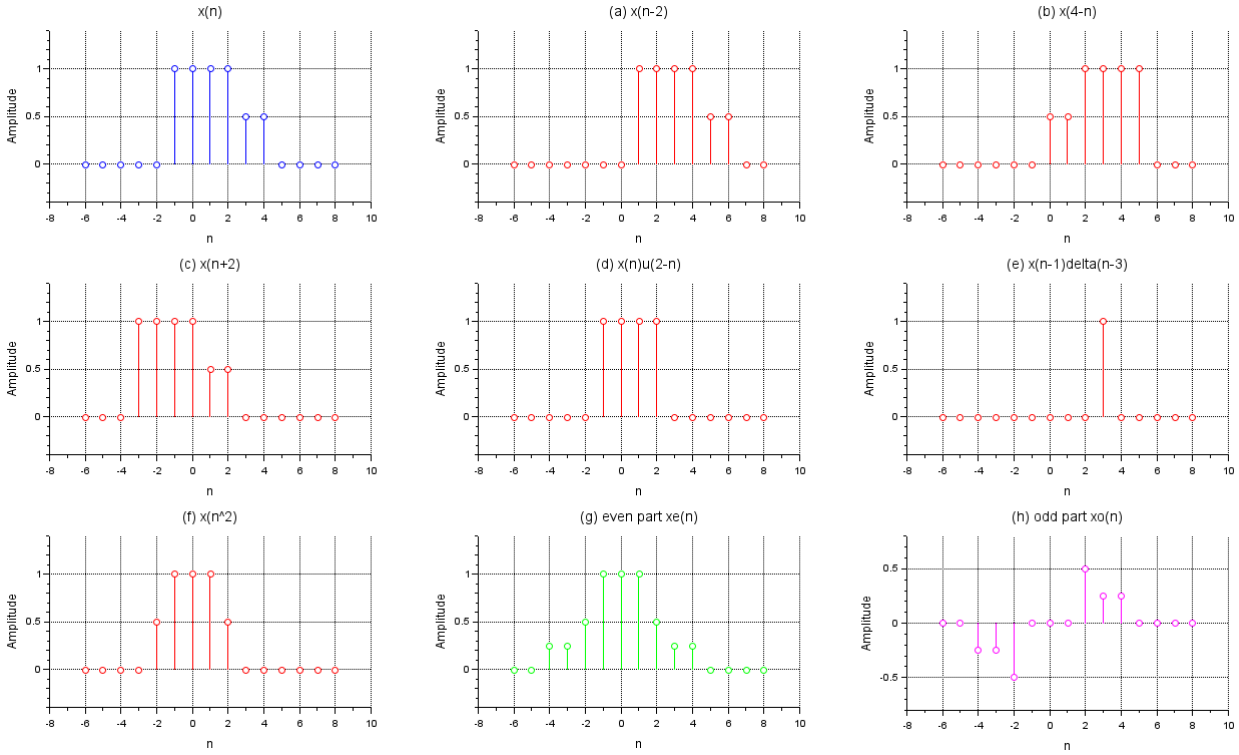


Figure 5: Additional exercise 2.2 – $x(n)$ and the signals (a)–(h)

The Scilab results agree with all the hand-derived sequences (a)–(h).

Additional exercise 2.4

Show that any signal can be decomposed into an even and an odd component. Is the decomposition unique? Illustrate your arguments using the signal

$$x(n) = \{2, 3, \underset{\uparrow}{4}, 5, 6\}$$

Answer:
Existence. For any signal $x(n)$, define

$$x_e(n) = \frac{1}{2}\big[x(n) + x(-n)\big], \qquad x_o(n) = \frac{1}{2}\big[x(n) - x(-n)\big]. \tag{10}$$

Then

- $x_e(-n) = \frac{1}{2}\big[x(-n) + x(n)\big] = x_e(n)$, so $x_e(n)$ is **even**;
- $x_o(-n) = \frac{1}{2}\big[x(-n) - x(n)\big] = -x_o(n)$, so $x_o(n)$ is **odd**;
- $x_e(n) + x_o(n) = \frac{1}{2}x(n) + \frac{1}{2}x(-n) + \frac{1}{2}x(n) - \frac{1}{2}x(-n) = x(n)$.

Hence every signal can be written as the sum of an even and an odd component.
Uniqueness. Suppose $x(n) = e(n) + o(n)$ for *some* even signal $e(n)$ and *some* odd signal $o(n)$. Replacing n by $-n$ and using $e(-n) = e(n)$, $o(-n) = -o(n)$:

$$\begin{cases} x(n) = e(n) + o(n) \\ x(-n) = e(n) - o(n) \end{cases} \implies e(n) = \frac{x(n) + x(-n)}{2} = x_e(n), \quad o(n) = \frac{x(n) - x(-n)}{2} = x_o(n).$$

Any even/odd pair that adds up to $x(n)$ must equal $x_e(n)$ and $x_o(n)$, so **the decomposition is unique**.

Illustration. With $x(n) = \{2, 3, \underset{\uparrow}{4}, 5, 6\}$ we have $x(-n) = \{6, 5, \underset{\uparrow}{4}, 3, 2\}$:

n	-2	-1	0	1	2
$x(n)$	2	3	4	5	6
$x(-n)$	6	5	4	3	2
$x_e(n)$	4	4	4	4	4
$x_o(n)$	-2	-1	0	1	2

$$x_e(n) = \{4, 4, \underset{\uparrow}{4}, 4, 4\}, \qquad x_o(n) = \{-2, -1, \underset{\uparrow}{0}, 1, 2\}$$

$x_e(n)$ is symmetric about $n = 0$, $x_o(n)$ is anti-symmetric with $x_o(0) = 0$, and their sum gives back $\{2, 3, \underset{\uparrow}{4}, 5, 6\}$. For example, the split $\{2, 3, \underset{\uparrow}{4}, 5, 6\} = \{2, 3, \underset{\uparrow}{4}, 3, 2\} + \{0, 0, \underset{\uparrow}{0}, 2, 4\}$ also sums to $x(n)$, but the second part is not odd. By the uniqueness proof, no other even/odd pair is possible.

Scilab: The function `evenodd(xn, xorigin)` uses the same (vector, origin) convention as the lab functions. It places $x(n)$ on a symmetric grid $[-L, L]$, so folding is just reversing the vector (`xs($:-1:1)`).

```
// ====================================================================
// Lab 3 - Additional Exercise 2.4
// Even / odd decomposition of x(n) = {2, 3, 4^, 5, 6}   (^ = origin)
// ====================================================================

// [xe, xo, yorigin] = evenodd(xn, xorigin)
//   xe(n) = [x(n) + x(-n)] / 2 ,  xo(n) = [x(n) - x(-n)] / 2
//   Both outputs are placed on a symmetric grid [-L, L] so that the
//   origin is exactly in the middle (yorigin = L + 1).
function [xe, xo, yorigin, xs, xf] = evenodd(xn, xorigin)
    nx = (1:length(xn)) - xorigin;     // time indices of x(n)
    L  = max(abs(nx));                 // half-length of symmetric grid
    xs = zeros(1, 2*L + 1);
    xs(nx + L + 1) = xn;               // x(n)  on [-L, L]
    xf = xs($:-1:1);                   // x(-n) on [-L, L]  (folding)
    xe = (xs + xf) / 2;
    xo = (xs - xf) / 2;
    yorigin = L + 1;
endfunction

function stem(n, x, ttl, c, spec)
    plot2d3(n, x, style=c);
    plot(n, x, spec);
    xtitle(ttl, "n", "Amplitude");
    a = gca();
    a.data_bounds = [min(n) - 1, min([x, 0]) - 1; max(n) + 1, max([x, 0]) + 1];
    xgrid();
endfunction

```

```
// ------------------------ Computation ----------------------------
[xe, xo, yorigin, xs, xf] = evenodd([2, 3, 4, 5, 6], 3);
n = (1:length(xe)) - yorigin;

mprintf("\n----- Exercise 2.4 -----\n");
mprintf("n       = %s\n", strcat(string(n),  "  "));
mprintf("x(n)    = %s\n", strcat(string(xs), "  "));
mprintf("x(-n)   = %s\n", strcat(string(xf), "  "));
mprintf("xe(n)   = %s\n", strcat(string(xe), "  "));
mprintf("xo(n)   = %s\n", strcat(string(xo), "  "));
mprintf("yorigin = %d\n", yorigin);
mprintf("xe even  (xe(n) =  xe(-n)) : %s\n", string(and(xe ==  xe($:-1:1))));
mprintf("xo odd   (xo(n) = -xo(-n)) : %s\n", string(and(xo == -xo($:-1:1))));
mprintf("xe + xo == x               : %s\n", string(and(xe + xo == xs)));

// -------------------------- Plotting -----------------------------
f24 = scf(); clf();
f24.figure_size = [1100, 800];
subplot(2, 2, 1); stem(n, xs, "x(n)",             2, "bo");
subplot(2, 2, 2); stem(n, xf, "x(-n)",            13, "ko");
subplot(2, 2, 3); stem(n, xe, "Even part xe(n)",  3, "go");
subplot(2, 2, 4); stem(n, xo, "Odd part xo(n)",   5, "ro");
```

Listing 5: Additional exercise 2.4 (`ex2_4_lab3.sce`)

Console output

```
n       = -2   -1   0   1   2
x(n)     = 2   3   4   5   6
x(-n)   = 6   5   4   3   2
xe(n)   = 4   4   4   4   4
xo(n)   = -2   -1   0   1   2
yorigin = 3
xe even  (xe(n) =  xe(-n)) :  T
xo odd   (xo(n) = -xo(-n)) :  T
xe + xo == x               :  T
```

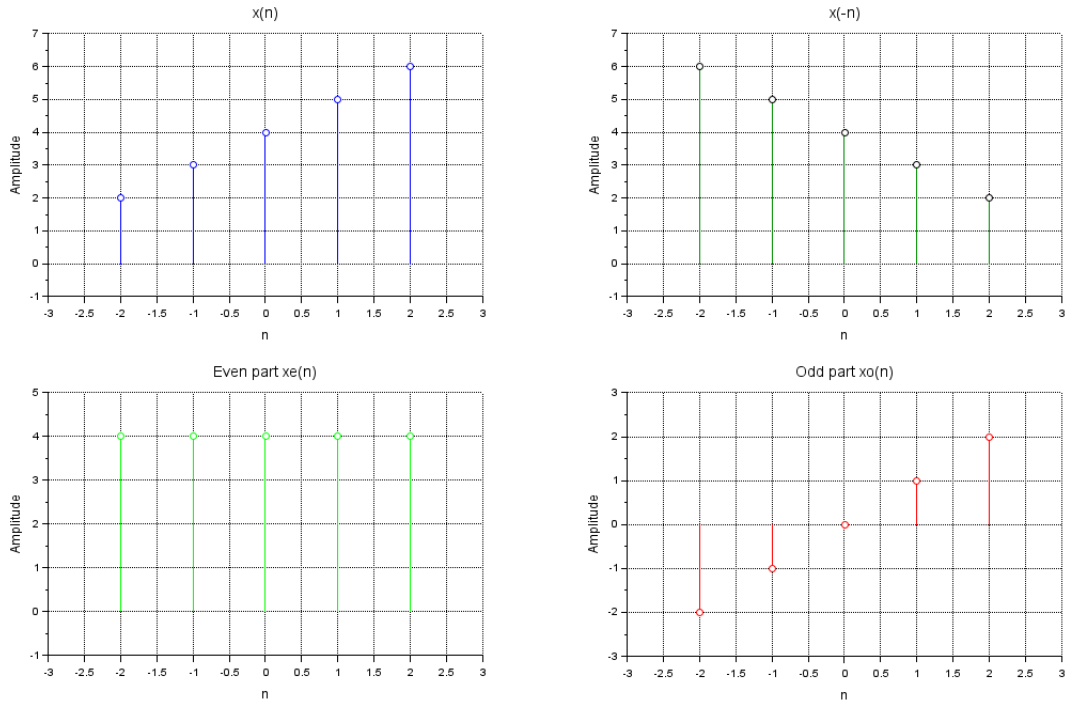


Figure 6: Additional exercise 2.4 – $x(n)$, $x(-n)$, even part $x_e(n)$ and odd part $x_o(n)$

Additional exercise 2.6

Consider the system $y(n) = \mathcal{T}[x(n)] = x(n^2)$.

(a) Determine if the system is time invariant.

(b) To clarify the result in part (a) assume that the signal $x(n) = \begin{cases} 1, & 0 \le n \le 3 \\ 0, & \text{elsewhere} \end{cases}$ is applied into the system. (1) Sketch the signal $x(n)$. (2) Determine and sketch the signal $y(n) = \mathcal{T}[x(n)]$. (3) Sketch the signal $y_2'(n) = y(n-2)$. (4) Determine and sketch the signal $x_2(n) = x(n-2)$. (5) Determine and sketch the signal $y_2(n) = \mathcal{T}[x_2(n)]$. (6) Compare the signals $y_2(n)$ and $y(n-2)$. What is your conclusion?

(c) Repeat part (b) for the system $y(n) = x(n) - x(n-1)$. Can you use this result to make any statement about the time invariance of this system? Why?

(d) Repeat parts (b) and (c) for the system $y(n) = \mathcal{T}[x(n)] = n\,x(n)$.

Answer:

A system is **time invariant** if shifting the input by k only shifts the output by the same k:

$$x(n) \xrightarrow{\mathcal{T}} y(n) \quad \Longrightarrow \quad x(n-k) \xrightarrow{\mathcal{T}} y(n-k), \quad \forall\, x(n), \forall\, k.$$

(a) For $y(n) = x(n^2)$, the response to the delayed input $x(n-k)$ is obtained by replacing n by n^2 *inside the argument of the input only*:

$$y(n,k) = \mathcal{T}[x(n-k)] = x(n^2-k),$$

while the delayed output is

$$y(n-k) = x\big((n-k)^2\big) = x(n^2 - 2nk + k^2).$$

Since $x(n^2-k) \neq x\big((n-k)^2\big)$ in general, the system is **time variant**.

(b) With $x(n) = \{\underset{\uparrow}{1}, 1, 1, 1\}$:

(1) $x(n) = \{\underset{\uparrow}{1}, 1, 1, 1\}$, $0 \le n \le 3$.

(2) $y(n) = x(n^2)$: $y(0) = x(0) = 1$, $y(\pm 1) = x(1) = 1$, $y(\pm 2) = x(4) = 0$, so

$$y(n) = \{1, \underset{\uparrow}{1}, 1\}$$

(3) $y_2'(n) = y(n-2) = \{\underset{\uparrow}{0}, 1, 1, 1\}$ (non-zero for $1 \le n \le 3$).

(4) $x_2(n) = x(n-2) = \{\underset{\uparrow}{0}, 0, 1, 1, 1, 1\}$ (non-zero for $2 \le n \le 5$).

(5) $y_2(n) = x_2(n^2) = x(n^2-2)$, which equals 1 only when $0 \le n^2 - 2 \le 3 \Leftrightarrow 2 \le n^2 \le 5 \Leftrightarrow n = \pm 2$:

$$y_2(n) = \{1, 0, \underset{\uparrow}{0}, 0, 1\}$$

(6) $y_2(n) = \{1, 0, \underset{\uparrow}{0}, 0, 1\} \neq y(n-2) = \{\underset{\uparrow}{0}, 1, 1, 1\}$. One counter-example is enough to show that the system is **time variant**, which confirms part (a).

(c) System $y(n) = x(n) - x(n-1)$:

(1) $x(n) = \{\underset{\uparrow}{1}, 1, 1, 1\}$.

(2) $y(n) = x(n) - x(n-1) = \{\underset{\uparrow}{1}, 0, 0, 0, -1\}$ ($y(0) = 1 - 0$, $y(4) = 0 - 1$).

(3) $y(n-2) = \{\underset{\uparrow}{0}, 0, 1, 0, 0, 0, -1\}$ (non-zero at $n = 2$ and $n = 6$).

(4) $x_2(n) = x(n-2) = \{\underset{\uparrow}{0}, 0, 1, 1, 1, 1\}$.

(5) $y_2(n) = x_2(n) - x_2(n-1) = \{\underset{\uparrow}{0}, 0, 1, 0, 0, 0, -1\}$.

(6) $y_2(n) = y(n-2)$ for this input.

Can we conclude time invariance from this result? *No.* Time invariance must hold for *every* input and *every* shift k. One example where $y_2(n) = y(n-2)$ is consistent with time invariance but does not prove it. (A single counter-example, on the other hand, is enough to prove time *variance*.) To prove it we need the general argument:

$$\mathcal{T}[x(n-k)] = x(n-k) - x(n-k-1) = y(n-k) \quad \forall\, x, k,$$

so the system $y(n) = x(n) - x(n-1)$ is indeed **time invariant**.

(d) System $y(n) = n\,x(n)$:

(1) $x(n) = \{\underset{\uparrow}{1}, 1, 1, 1\}$.

(2) $y(n) = n\,x(n) = \{\underset{\uparrow}{0}, 1, 2, 3\}$.

(3) $y(n-2) = (n-2)\,x(n-2) = \{\underset{\uparrow}{0}, 0, 0, 1, 2, 3\}$ (values $0, 1, 2, 3$ at $n = 2, \ldots, 5$).

(4) $x_2(n) = x(n-2) = \{\underset{\uparrow}{0}, 0, 1, 1, 1, 1\}$.

(5) $y_2(n) = n\,x_2(n) = \{\underset{\uparrow}{0}, 0, 2, 3, 4, 5\}$.

(6) $y_2(n) \neq y(n-2)$ (e.g. $y_2(2) = 2$ but $y(0) = 0$), so the system is **time variant**.

Statement about time invariance (part (c) for this system): here the example *is* enough to decide, because one counter-example proves time variance. In general,

$$\mathcal{T}[x(n-k)] = n\,x(n-k) \;\neq\; y(n-k) = (n-k)\,x(n-k).$$

The gain n depends explicitly on time, so the system is time variant.

Table 2: Summary of Additional exercise 2.6

System	$y_2(n)$ vs. $y(n-2)$	Conclusion
$y(n) = x(n^2)$	different	Time variant
$y(n) = x(n) - x(n-1)$	equal	Time invariant (proved analytically)
$y(n) = n\,x(n)$	different	Time variant

Scilab: Each system is written as a function `T(f, n)` that receives the *input signal as a function* `f` and returns the output at the indices `n`. This way, $y(n-2)$ is simply `T(xin, n-2)` and $y_2(n) = \mathcal{T}[x(n-2)]$ is `T(x2in, n)`. The function `check_system` performs steps (1)–(6) for each system. Step (6) is plotted as the difference $y_2(n) - y(n-2)$: if it is identically zero, the shift of the input only shifts the output.

```
// ====================================================================
// Lab 3 - Additional Exercise 2.6 : time-invariance test
//   x(n) = 1 for 0 <= n <= 3, 0 elsewhere
//   System 1: y(n) = x(n^2)
//   System 2: y(n) = x(n) - x(n-1)
//   System 3: y(n) = n x(n)
// A system T is evaluated as T(f, m): output y(m) when input is f(.)
// ====================================================================

function y = xin(n)                  // input x(n)
    y = bool2s(n >= 0 & n <= 3);
endfunction

function y = x2in(n)                 // delayed input x2(n) = x(n-2)
    y = xin(n - 2);
endfunction

function y = T1(f, n)                // y(n) = x(n^2)
    y = f(n.^2);
endfunction
```

```
21
22 function y = T2(f, n)                // y(n) = x(n) - x(n-1)
23     y = f(n) - f(n - 1);
24 endfunction
25
26 function y = T3(f, n)                // y(n) = n x(n)
27     y = n .* f(n);
28 endfunction
29
30 function stem(n, x, ttl, c, spec)
31     plot2d3(n, x, style=c);
32     plot(n, x, spec);
33     xtitle(ttl, "n", "Amplitude");
34     a = gca();
35     a.data_bounds = [min(n) - 0.5, min([x, 0]) - 0.5; max(n) + 0.5, max([x, 0]) +
       0.5];
36     xgrid();
37 endfunction
38
39 function show(name, n, x)
40     k  = find(x <> 0);
41     lo = min([n(k), 0]);
42     hi = max([n(k), 0]);
43     idx = find(n >= lo & n <= hi);
44     s = string(x(idx));
45     s(n(idx) == 0) = s(n(idx) == 0) + "*";
46     mprintf("  %-14s = {%s},   n = %d..%d\n", name, strcat(s, ", "), lo, hi);
47 endfunction
48
49 // Steps (1)-(6) of part (b) for a given system T
50 function f = check_system(T, label, n)
51     x   = xin(n);           // (1) x(n)
52     y   = T(xin, n);        // (2) y(n)  = T[x(n)]
53     yd  = T(xin, n - 2);    // (3) y(n-2)
54     x2  = x2in(n);          // (4) x2(n) = x(n-2)
55     y2  = T(x2in, n);       // (5) y2(n) = T[x2(n)]
56     dif = y2 - yd;          // (6) compare y2(n) with y(n-2)
57
58     mprintf("\n----- %s -----\n", label);
59     show("x(n)", n, x);
60     show("y(n)", n, y);
61     show("y(n-2)", n, yd);
62     show("x2(n)=x(n-2)", n, x2);
63     show("y2(n)=T[x2]", n, y2);
64     if and(dif == 0) then
65         mprintf("  => y2(n) = y(n-2) for this input (consistent with time-invariance
       )\n");
66     else
67         mprintf("  => y2(n) <> y(n-2)  ==> the system is TIME-VARIANT\n");
```

```
    end

    f = scf(); clf();
    f.figure_size = [1100, 900];
    subplot(3, 2, 1); stem(n, x,   "(1) x(n)",                  2, "bo");
    subplot(3, 2, 2); stem(n, y,   "(2) y(n) = T[x(n)]",        5, "ro");
    subplot(3, 2, 3); stem(n, yd,  "(3) y(n-2)",                5, "ro");
    subplot(3, 2, 4); stem(n, x2,  "(4) x2(n) = x(n-2)",        2, "bo");
    subplot(3, 2, 5); stem(n, y2,  "(5) y2(n) = T[x2(n)]",      3, "go");
    subplot(3, 2, 6); stem(n, dif, "(6) y2(n) - y(n-2)",        6, "mo");
endfunction

// ------------------------ Run the tests --------------------------
n = -4:8;
f261 = check_system(T1, "System 1: y(n) = x(n^2)", n);
f262 = check_system(T2, "System 2: y(n) = x(n) - x(n-1)", n);
f263 = check_system(T3, "System 3: y(n) = n x(n)", n);
```

Listing 6: Additional exercise 2.6 (`ex2_6_lab3.sce`)

Console output

```
(* marks the sample at n = 0)
--- System 1:  y(n) = x(n^2) ---
x(n)        = {1*, 1, 1, 1},    n = 0..3
y(n)        = {1, 1*, 1},    n = -1..1
y(n-2)      = {0*, 1, 1, 1},    n = 0..3
x2(n)=x(n-2) = {0*, 0, 1, 1, 1, 1},    n = 0..5
y2(n)=T[x2]  = {1, 0, 0*, 0, 1},    n = -2..2
⇒ y2(n) ≠ y(n-2) ⟹ the system is TIME-VARIANT
--- System 2:  y(n) = x(n) - x(n - 1) ---
x(n)        = {1*, 1, 1, 1},    n = 0..3
y(n)        = {1*, 0, 0, 0, -1},    n = 0..4
y(n-2)      = {0*, 0, 1, 0, 0, 0, -1},    n = 0..6
x2(n)=x(n-2) = {0*, 0, 1, 1, 1, 1},    n = 0..5
y2(n)=T[x2]  = {0*, 0, 1, 0, 0, 0, -1},    n = 0..6
⇒ y2(n) = y(n-2) for this input (consistent with time-invariance)
--- System 3:  y(n) = n x(n) ---
x(n)        = {1*, 1, 1, 1},    n = 0..3
y(n)        = {0*, 1, 2, 3},    n = 0..3
y(n-2)      = {0*, 0, 0, 1, 2, 3},    n = 0..5
x2(n)=x(n-2) = {0*, 0, 1, 1, 1, 1},    n = 0..5
y2(n)=T[x2]  = {0*, 0, 2, 3, 4, 5},    n = 0..5
⇒ y2(n) ≠ y(n-2) ⟹ the system is TIME-VARIANT
```

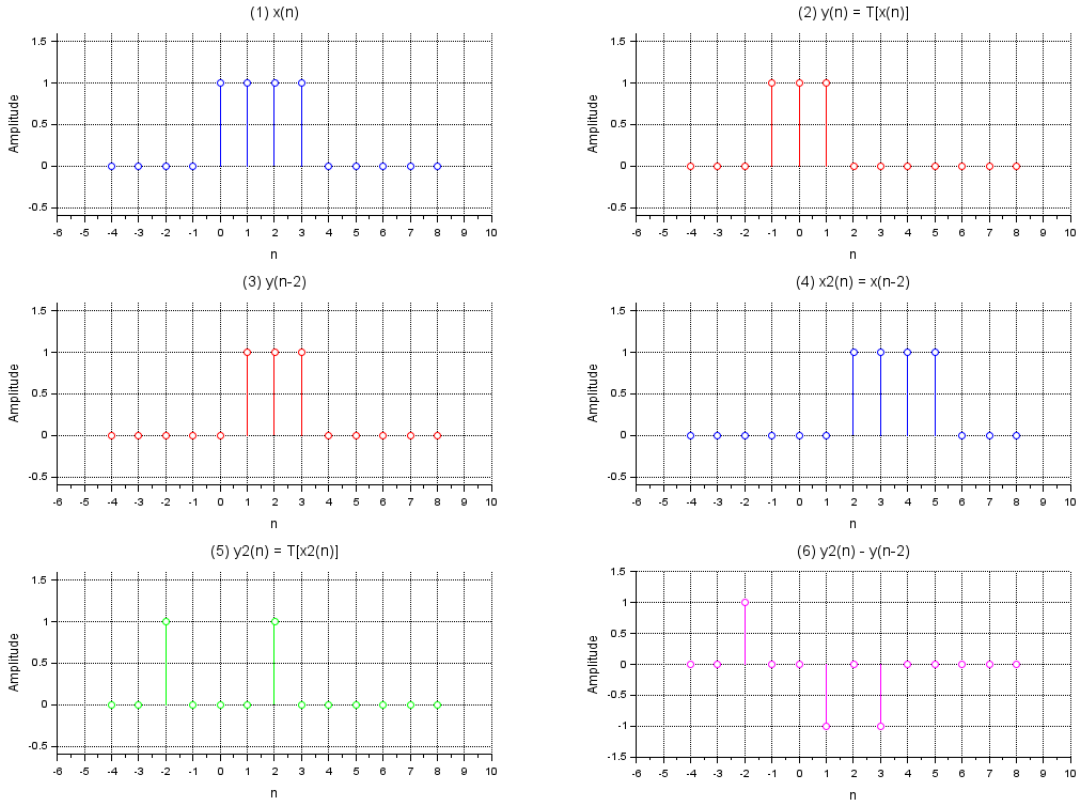


Figure 7: Additional exercise 2.6(b) – system $y(n) = x(n^2)$: $y_2(n) - y(n-2) \neq 0 \Rightarrow$ time variant

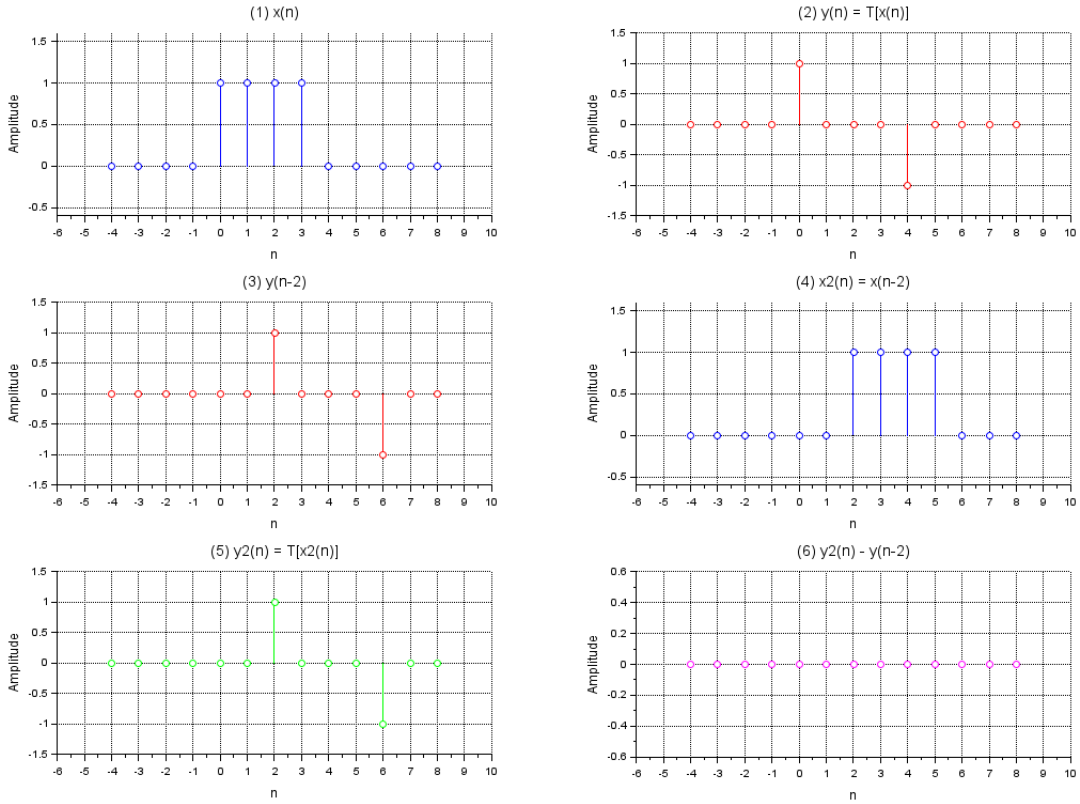


Figure 8: Additional exercise 2.6(c) – system $y(n) = x(n) - x(n-1)$: $y_2(n) - y(n-2) = 0$

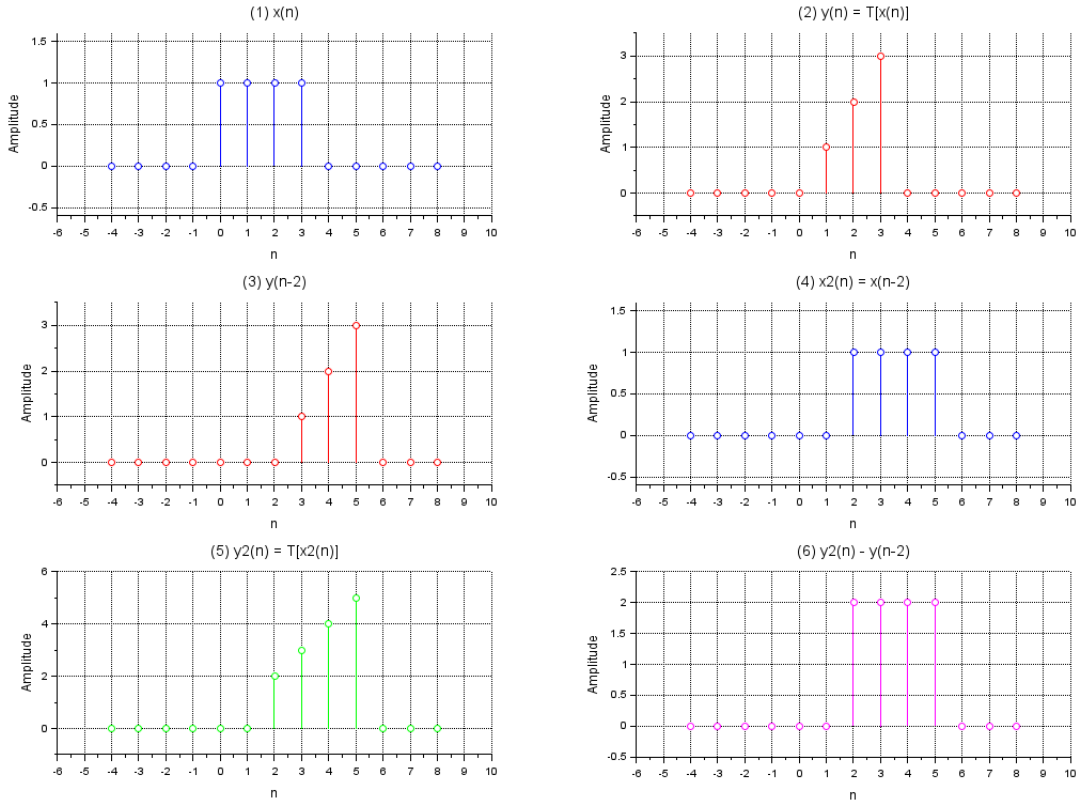


Figure 9: Additional exercise 2.6(d) – system $y(n) = n\,x(n)$: $y_2(n) - y(n-2) \neq 0 \Rightarrow$ time variant

5 Discussion

Representing a signal by the pair (vector, origin) made it possible to implement every time-domain operation as a general Scilab function. The exercises show that most operations only need careful index bookkeeping:

- Time shifts (`advance`, `delay`) leave the sample vector unchanged and only move the origin. Zero-padding is needed only when the origin falls outside the vector.
- Sample-by-sample operations (`add`, `multi`) require both signals to be aligned on a common index range by zero-padding before using the element-wise operators.
- For `convolution`, the built-in `conv` gives the sample values, and the origin follows from $n_{y,min} = n_{x,min} + n_{h,min}$.

In the additional exercises, evaluating a signal as a function of an arbitrary index vector turned out to be a convenient way to compute folded, shifted or non-linearly indexed signals such as $x(n^2)$. Exercise 2.6 also shows that one example can only *disprove* time invariance. Proving it requires a general argument.

6 References

- J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed., Pearson, 2007 – Chapter 2 Problems.
- Digital Signal Processing Lecture Notes - Ho Chi Minh City University of Technology.
- Scilab Online Manuals: https://help.scilab.org/doc/5.5.2/en_US/function.html, https://help.scilab.org/doc/5.5.2/en_US/deff.html.

7 Source Code

The source code, scripts, and LaTeX files for this laboratory report can be accessed at our GitHub repository:
https://github.com/aulequangtri/signal_scilab